HOCHSCHULE RAVENSBURG-WEINGARTEN

PROF. DR. ING. S. KELLER

STUDIENGANG ANGEWANDTE INFORMATIK

GRAFISCHE BEDIENOBERFLÄCHEN IN JAVA SWING

Beschreibung einer grafischen Oberfläche

zur Vorbereitung auf die Klausur Grabo

im WS 2007/08

Ausgeteilt in der Vorlesung **Mo 21.1.2008**

Realisierung einer grafischen Oberfläche unter Java swing

Zu realisieren ist eine Java-Standalone-Anwendung mit GUI unter Verwendung von swing-Komponenten. Die Java-Anwendung realisiert ein einfaches bekanntes Verschiebe-Spiel. Die folgenden Bilder visualisieren die zu realisierende Oberfläche.

Beim Starten der Anwendung erscheint ein Fenster mit dem Titel „Verschiebespiel". Das Look&Feel der Oberfläche ist Metal (Voreinstellung in Java) *(siehe Bild links).*

Das Fenster stellt ein Spielfeld mit 3 Zeilen und 3 Spalten dar. Das Spielfeld enthält 8 verschiebbare Spielbausteine. Die Spielbausteine enthalten die Ziffern 1 – 8. Die Bausteine mit den Ziffern sind zufällig im Spielfeld positioniert. Das letzte Feld rechts unten ist leer. Die zufälligen Positionen der Ziffern sind bei jedem neuen Spielstart verschieden, so dass der Spieler bei jedem Spielstart eine neue Ausgangssituation vorfindet. Jeder Baustein hat eine Größe von 70 x 70 Pixel. Die Ziffern sollen mit der Schrift Arial,18 Punkt, fett und in schwarz dargestellt werden.

Das Fenster wird durch pack() in seiner **optimalen Größe** dargestellt.

In das leere Feld kann ein benachbarter Baustein durch Anwahl mit der Maus verschoben werden. Das Bild links zeigt einen Zustand, bei dem mehrerer Verschiebungen stattgefunden haben. In dieser dargestellten Situation kann die Ziffer 1 nach links, die Ziffer 7 nach rechts und die Ziffer 5 nach unten geschoben werden. Die Anwahl einer anderen Ziffer bewirkt keine Reaktion, da kein Nachbarfeld frei ist.

Ziel des Spieles ist, alle Bausteine so zu verschieben, dass von links nach rechts und oben nach unten gelesen die Ziffernfolge aufsteigend sortiert ist also 1,2,3,4,5,6,7,8. Das letzte Feld muss leer sein. Dieser Zustand **„gewonnen"** wird durch eine modale Dialogbox angezeigt.

In dieser Dialogbox kann der Anwender entscheiden, ob er ein neues Spiel beginnen will oder das Spiel beenden möchte *(siehe Bild rechts)*. Bei Anwahl von „Ja" wird ein neues Spiel begonnen, bei „Nein" wird das Spiel beendet.

Die Farben der Spielbausteine kann über den Menüpunkt „Farbe ändern" in der Menüleiste geändert werden.

Sie sollen eine Farbauswahl für zwei von Ihnen frei definierbaren Farben und als dritten Wahlmöglichkeit die voreingestellte Farbe zu Verfügung stellen.

Das Bild rechts zeigt als Beispiel das Menü mit den Menüpunkten rot, blau und voreinstellung.

Das Bild rechts zeigt das Spiel nach Anwahl der Farbe rot.

Zur Implementierung sollen Sie die in der Vorlesung eingeführte Architektur **„3-Schichtenmodell"** einhalten. Das bedeutet, dass Sie **mehrere Klassen implementieren** müssen.

Die **Anwendungsschicht** soll durch genau **eine** Klasse „ **logik**" realisiert werden. In dieser Klasse sollen folgende Methoden zu Verfügung gestellt werden:

Konstruktor: logik

`public logik()`

Initialisiert ein array von Integer-Objekten. Die Ziffern 1 - 8 werden an zufällig errechneten Positionen gespeichert, so dass man eine unsortierte Liste erhält. Der letzte Wert im array ist zu Beginn „null".

Die Klasse enthält einen statischen Zähler, der die Anzahl der Verschiebungen zählt. Im Konstruktor wird dieser Zähler auf 0 gesetzt.

Methode getNummern()

`public java.lang.String[] getNummern()`

Die Methode getNummer() liefert die Ziffern des arrays von links nach rechts als Liste von Strings. Das leere Feld wird durch einen leeren String "" codiert

Returns:

Liste mit Strings, die die Ziffern der Bausteine repräsentieren. Das leere Feld wird als Leerstring "" geliefert.

Methode getZaehler()

`public int getZaehler()`

Die Methode getZaehler() liefert den aktuellen Zählerstand

Returns: aktuellen Zählerstand

Methode move()

`public boolean move(java.lang.String ziffer)`

Die Methode move() verschiebt die 'ziffer', falls das Nachbarfeld leer ist

Parameters: ziffer - zu verschiebende Ziffer

Returns: true, wenn eine Verschiebung durchgeführt wurde
false, wenn keine Verschiebung stattgefunden hat

Methode richtigeReihenfolge()

`public boolean richtigeReihenfolge()`

Die Methode richtigeReihenfolge() prüft, ob die Ziffern in der richtigen Reihenfolge
→ 1,2,3,4,5,6,7,8,null in der Liste stehen

Returns: true , falls die richtige Reihenfolge erkannt wurde, sonst false